

What a BNF Grammar Defines

A BNF grammar is a *recipe* for building strings.

- Start at the **start symbol** (the first **non-terminal** rule)
- Repeatedly replace a **non-terminal** using a rule (a **production**)
- Stop when you have only **terminals**

The **language** of the grammar is the set of *all* strings you can build this way.

BNF Grammars

Sam Ogden

Updated 2026-04-21

Motivating Example

Writing a JSON Parser

```
{  
  "1": {  
    "location": "assign-ostep7",  
    "weight": 20  
  },  
  "2": {  
    "location": "assign-ostep8",  
    "weight": 20  
  },  
  "3": {  
    "location": "assign-msh3",  
    "weight": 60  
  },  
}
```

(example input text)

Someone asks you to build a JSON parser. How do you approach it?

Input. A blob of text *Output.* A dictionary **or** error

What is a “language”?

A language is a set of strings

- {"a", "ab", "abc"}
- {"x=1", "x=y"}

Infinite languages can't be defined by listing all their strings

- {"a", "ab", ...}

But we have to be careful because these aren't *explicit* definitions

Is the next string "aba" or "abc"?

We need a better way to define languages

Backus-Naur Form (BNF) Grammars

$$\begin{array}{lcl} R_1 & ::= & a \mid aR_2 \\ R_2 & ::= & b \mid bR_1 \end{array}$$

Three parts of grammars:

1. **non-terminal** *symbols* are placeholders that *must* be replaced
2. **terminals** are concrete parts of the output string
3. **productions** are what we replace **non-terminals** with
 - e.g. “**bR₁**” in the above example

Using BNFs

To use a BNF:

1. Start with a the start symbol
 - This is the first listed **non-terminal**
2. Replace any **non-terminal** with one of its **productions**
3. Repeat #2 until we have no **non-terminal** symbols left

$$\begin{array}{lcl} R_1 & ::= & a \mid aR_2 \\ R_2 & ::= & b \mid bR_1 \end{array}$$

BNF Practice

Let's try generating some strings

```
 $R_1 ::= \text{foo} \mid \text{bar}$ 
```


Let's try generating some strings

$R_1 ::= [R_2]$

$R_2 ::= a \mid b \mid c$

Let's try generating some strings

$R_1 ::= aR_1b$

Let's try *parsing* some strings

$R_1 ::= aR_1b$

aabb

Let's try *parsing* some strings

$R_1 ::= aR_1b$

abb

Let's try *parsing* some strings

$R_1 ::= R_1 + R_1$

abb

Let's try *parsing* some strings

$R_1 ::= aR_1b \mid 1 \mid x$

$1 + x + x$

Let's try *parsing* some strings

$R_1 ::= aR_1b \mid 1 \mid x$

$1 + x +$